

# The MVL Research Program

Trustworthy Software When Machines Write the Code

Ilja Heitlager — Schuberg Philis / TU Eindhoven

April 2026

## The MVL Research Program

### Why this work exists

For sixty years, programming languages have been designed for humans to write. That era is ending. Large language models generate code in any language, at any verbosity, with any annotation burden — for free. The question is no longer “how do we write better code?” but “**how do we trust code we didn’t write?**”

This is not a theoretical concern. At Schuberg Philis — a Dutch IT company running mission-critical infrastructure for clients including NS (Dutch Railways), Port of Rotterdam, and Tennet (national grid operator) — we deploy AI-assisted engineering daily. Our engineers use LLMs to generate code, specifications, and tests. The productivity gains are real. But so is the accountability: when an LLM-generated change breaks a train scheduling system or a power grid interface, the question from the regulator is not “which model generated this?” but “where is your evidence that this code satisfies its specification?”

We developed **Assured Agentic Engineering (AAE)** as a maturity framework with five levels, from responsible AI usage (AAE-1) through full external certification (AAE-5). Our target: every engineer at AAE-3 (spec-centric, evidence-linked) by summer 2026. But AAE-3 is a process answer. It tells you *how* to work. It doesn’t tell you *what* the compiler should prove.

That is the research gap this program addresses.

### The inversion

If LLMs generate all code, every design decision about programming languages inverts:

- **Writability becomes irrelevant.** Syntactic sugar, list comprehensions, operator overloading — all exist because humans are slow typists. LLMs are not.
- **Annotation burden becomes free.** Termination proofs, security labels, refinement predicates — prohibitively expensive for humans, zero cost for machines.
- **Verification becomes the bottleneck.** When generation is instant, the only remaining cost is proving correctness. Language design should maximize what the compiler can prove per token.

This inversion is the foundation of the MVL — the Minimum Verification Language — and the research program around it.

## From practice to theory

This research grows directly from industrial practice. The models, requirements, and proofs are not speculative — they are formalized observations of what works (and what fails) in mission-critical AI-assisted development at scale. The progression from AAE-3 (process assurance) to AAE-5 (compiler-based certification) is not a vision statement. It is an engineering roadmap with implementations, specifications, and a working compiler.

## The research program

Five papers, each building on the previous. Together they form a complete argument: from trust model to language design to architectural principles.

---

### Paper 1: The ISPE Model

**Working title:** *The ISPE Model: A Trust-Based Framework for Software System Evolution in the LLM Era*

We introduce Intent-Specification-Program-Evidence as a formal framework for reasoning about trust in incrementally evolving software systems. We prove that system trust is the mathematical product — not sum — of the trust of individual changes, derive a calculus for trust evolution, and validate the model through independent convergence with MISRA C, DO-178C, and ISO 15026. The framework applies regardless of code origin — human or LLM-generated.

This paper provides the theoretical foundation. It is what makes AAE-3 (assured agentic engineering) formally definable: completeness, coverage, and assurance become measurable KPIs rather than checklists.

**Venue:** IEEE Transactions on Software Engineering (TSE) or Journal of Systems and Software (JSS) **Repo:** `~/papers/ispe_model/` — draft advanced, sections written **Co-author:** Patricia Lago (VU Amsterdam) — proposed

---

### Paper 2: Eleven Requirements for Trustworthy Code Generation

**Working title:** *Eleven Requirements for Trustworthy Code Generation: What a Programming Language Must Enforce When Machines Write the Code*

We derive eleven requirements that a target language must enforce for generated code to be trustworthy through evolution. Seven emerge from the convergence of formal methods (Curry-Howard, linear logic, algebraic effects) and industrial failure analysis (MISRA C, DO-178C, IEC 61508). Four more become feasible when LLMs eliminate the human annotation cost. We score seven mainstream languages — none exceeds 6 of 11.

This paper answers the question Paper 1 leaves open: if P (Program) is inevitable in the trust chain, what properties must P have? The scorecard is falsifiable and immediately applicable to any code generation pipeline.

**Venue:** ICSE, ESEC/FSE, or IEEE TSE **Repo:** `~/papers/language_requirements/` — scaffolded, all sections TODO

---

### Paper 3: Language Contraction for Verified Code Generation

**Working title:** *Language Contraction for Verified Code Generation: Designing a Minimal Target Language for LLM-Produced Software*

We present a language contraction — sixteen features systematically removed rather than added — resulting in a minimal language of ~25 keywords with an LL(1) grammar that enforces all eleven requirements at compile time. We introduce the corpus hypothesis (LLM generation quality correlates with  $\log(\text{corpus size})$ , with three exploitable exceptions) and demonstrate through code examples that the contracted language achieves equivalent expressiveness to Rust for safety-critical domains with stronger guarantees.

This is the engineering paper: not just what requirements exist, but how to build a language that satisfies all of them. It demonstrates the path from AAE-3 (process assurance) toward AAE-4/5 (compiler-based assurance) where the compiler itself generates the evidence that certification requires.

**Venue:** OOPSLA, PLDI, or ICFP **Repo:** `~/papers/language_design/` — scaffolded, all sections **TODO** **Implementation:** `github.com/LAB271/mv1_language` — v0.5.1, parser + type checker, 215 tests

---

### Paper 4: Architecture Principles for the LLM Era

**Working title:** *From Comprehension to Trust: Software Architecture When Code Is Free*

We derive ten architecture principles from the observation that LLMs make code generation frictionless but do not make trust free. Old architecture optimized for human comprehension (abstractions, layers, modules). New architecture optimizes for trust boundaries, blast radius, sovereignty, and regeneration. We validate against MISRA C, Anthropic’s engineering rules, and four industrial case studies.

This paper extends the ISPE model to architectural decisions. It provides the theoretical backing for why agentic engineering requires different architectural patterns than traditional development — and what those patterns are.

**Venue:** ICSA (International Conference on Software Architecture) or JSS **Research material:** exists in knowledge base, no repo yet

---

### Paper 5: Empirical Validation — The MVL Compiler as Proof of Concept

**Working title:** *Compiling Trust: Empirical Evaluation of an Eleven-Requirement Language for LLM Code Generation*

We present empirical results from the MVL compiler: generation quality benchmarks (LLM-generated MVL vs. Python vs. Rust on identical specifications), compilation success rates, requirement satisfaction rates, and assurance report quality. We test the corpus hypothesis quantitatively and measure whether the contracted language design actually improves LLM output trustworthiness.

This is the validation paper — it takes the theoretical claims of Papers 1-4 and tests them against reality. Depends on the MVL compiler reaching Phase 1 completion (.mvl to native binary with all 11 requirements enforced).

**Venue:** ICSE (empirical track), ASE, or ESEC/FSE **Depends on:** MVL compiler Phase 1 completion

---

## The arc

|          |                          |                                   |                |
|----------|--------------------------|-----------------------------------|----------------|
| Paper 1: | What is trust?           | → ISPE model, trust product       | (theory)       |
| Paper 2: | What must P look like?   | → 11 requirements, scorecard      | (requirements) |
| Paper 3: | How to build P?          | → Language contraction, MVL       | (design)       |
| Paper 4: | How to architect for it? | → 10 principles, trust boundaries | (architecture) |
| Paper 5: | Does it work?            | → Benchmarks, empirical evidence  | (validation)   |

Papers 1-3 form a self-contained trilogy. Papers 4-5 extend and validate.

## Why this matters for the collaboration

This research program sits at the intersection of software engineering, programming language design, and AI-assisted development. It is grounded in industrial practice (mission-critical systems at Schuberg Philis) and contributes both theory (trust product, ISPE model) and artifacts (MVL compiler, assurance tooling).

For a collaborating research group, this offers:

- **Student projects** — the MVL compiler has 40+ open issues ranging from type system extensions to LLVM backends
- **Empirical research** — real industrial data from AAE adoption at a company with 300+ engineers
- **Publication pipeline** — five papers with clear scopes, working titles, and target venues
- **A running system** — not a proposal, but a compiler (v0.5.1, 215 tests, CI, specifications)
- **Cross-domain reach** — connects to software architecture (Lago), program analysis (Zaidman), and formal methods

The question we are asking is simple: if machines write all the code, what should the programming language look like? The answer turns out to be smaller, not larger — and provably so.